
Como o Leafcutter foi arquitetado

Uma plataforma de integração confiável construída com Elixir, Phoenix, OTP e Broadway

Leafcutter Architecture Baseline

25 de agosto de 2026

1	Introdução	11	Interceptor
1.1	Estado do documento	12	O modelo de execução
2	O caso mais simples	12.1	Run supervision tree
3	Princípios que orientam o desenho	12.2	RunCoordinator
3.1	Usar a menor primitive correta	12.3	Broadway como data plane
3.2	Funções puras no centro, efeitos nas bordas	13	Fan-out durável
3.3	Estado operacional e estado durável	13.1	Por que não usar fila externa primeiro
3.4	Contexts isolados	14	Destination pipelines
4	As três unidades fundamentais: Package, Integration e Run	14.1	Claim
4.1	Integration Package	14.2	Retry
4.2	Integration	15	Error model
4.3	Run	16	At-least-once
5	Catalog	17	Checkpoint
6	Contracts com JSON Schema	18	Run ownership e recovery entre nodes
6.1	Por que validar as duas pontas	18.1	Um Run, um node
6.2	Contratos versionados	18.2	Heartbeat por node
6.3	Compilação	18.3	Recovery
7	Connector, Operation e Transport	19	O cluster BEAM
7.1	Connector	19.1	Infraestrutura inicial
7.2	Read Operation	19.2	Distributed Erlang
7.3	Write Operation	19.3	Evolução
7.4	Transport	20	Connections e Secrets
8	Source Identity, Payload Hash e IdentityMapping	20.1	Connection
8.1	Record ID	20.2	Secret
8.2	Source Identity	21	Environments, homologação e promoção
8.3	Payload Hash	22	RBAC
8.4	IdentityMapping	23	API-first
9	Transformation		
10	Enrichment		

24 Monitoring, observabilidade e auditoria

- 24.1 Execution Monitoring
- 24.2 Platform Observability
- 24.3 Attempt
- 24.4 ExecutionEvent
- 24.5 AuditEvent
- 24.6 Notifications

25 Storage e retenção

26 Phoenix Contexts

- 26.1 Por que não um context por tabela
- 26.2 Por que Executions não vira god module

27 Apps da umbrella

- 27.1 Core
- 27.2 Connectors
- 27.3 Runtime
- 27.4 API

28 Documentação como parte da arquitetura

29 Harness e continuidade

30 Roadmap

- 30.1 V0 - Foundation
- 30.2 V1 - Reliable HTTP core
- 30.3 V1.x - Reliability
- 30.4 V2 - Governance
- 30.5 V3 - Ecosystem
- 30.6 V4 - Inbound e multi-transport
- 30.7 V5+

31 O que ainda precisa ser ratificado

32 Conclusão

1 Introdução

O Leafcutter é uma plataforma para construir e operar integrações entre sistemas.

A ideia central é simples:

```
System A
  ↓ extract once
validated data
  ├── transform for System B → batch → deliver to B
  └── transform for System C → batch → deliver to C
```

A simplicidade do desenho não elimina os problemas reais. Sistemas externos possuem formatos diferentes, autenticação, paginação, rate limits, respostas parciais, indisponibilidade e IDs incompatíveis. Uma execução pode atravessar milhões de registros e vários destinos. Um node pode cair no meio do processo. Um destination pode permanecer fora do ar enquanto os demais continuam saudáveis.

A arquitetura do Leafcutter foi desenhada para lidar com esses problemas sem transformar o projeto em uma coleção de frameworks internos. O objetivo é usar as primitivas corretas do ecossistema Elixir e manter o código compreensível para um desenvolvedor que deseja dominá-lo ponta a ponta.

Este documento explica as peças do Leafcutter, a responsabilidade de cada uma e como elas trabalham juntas.

1.1 Estado do documento

Esta é uma baseline arquitetural. Ela separa:

- decisões já aceitas;
- propostas que ainda serão ratificadas context por context;
- evoluções futuras que não devem influenciar a implementação inicial.

A arquitetura não deve ser tratada como desculpa para antecipar código. O repositório cresce por vertical slices pequenos, documentados e testáveis.

2 O caso mais simples

Considere uma empresa que possui clientes em um ERP e precisa distribuí-los para um CRM e um sistema financeiro.

O ERP entrega:

```
{
  "id": 42,
  "name": "Maria Silva",
  "age": 17,
  "email": "maria@example.com",
  "credit_limit": 5000
}
```

O CRM espera:

```
{
  "external_id": "42",
  "full_name": "Maria Silva",
  "access_allowed": false
}
```

O sistema financeiro espera:

```
{
  "customer_code": 42,
  "customer_name": "Maria Silva",
  "credit_limit": 5000
}
```

O Leafcutter extrai Maria uma vez e cria duas responsabilidades independentes:

```
ERP Customer 42 — [ CRM
                  [ Billing
```

Cada branch possui seu próprio contrato, Transformation, batching, rate limit, concorrência, retries e histórico.

Se Billing estiver fora do ar:

```
CRM      completed
Billing   pending / retrying
```

CRM continua. Billing volta a processar quando estiver disponível. O dado não precisa ser buscado novamente do ERP apenas porque um destino falhou.

Esse exemplo pequeno contém quase toda a arquitetura.

3 Princípios que orientam o desenho

3.1 Usar a menor primitive correta

A ordem de preferência é:

```
Elixir/Erlang standard library
→ OTP
→ Phoenix/Ecto/PubSub/Broadway/Oban
→ plain module/function
→ custom abstraction only when necessary
```

Isso não significa usar OTP em tudo. Um processo só existe quando há estado ao longo do tempo, lifecycle, mensagens, coordenação, concorrência ou isolamento de falha.

Transformar um payload é uma função. Controlar o lifecycle de um Run é um problema OTP. Mover milhões de mensagens com demand e batching é um problema Broadway. Guardar checkpoint é um problema de persistência.

3.2 Funções puras no centro, efeitos nas bordas

A lógica específica de integração deve ser fácil de testar:

```
@spec transform(map(), map()) ::
  {:ok, map()}
  | {:ok, [map()]}
  | :skip
  | {:error, term()}
def transform(customer, config) do
  {:ok,
   %{
     "external_id" => Integer.to_string(customer["id"]),
     "full_name" => customer["name"],
     "access_allowed" => customer["age"] >= config["minimum_age"]
   }}
end
```

Essa função não conhece HTTP, Repo, Oban, PubSub ou secrets.

3.3 Estado operacional e estado durável

OTP/Broadway
→ o que está acontecendo agora

PostgreSQL
→ de onde podemos continuar se tudo desaparecer

Supervisor reinicia um processo. PostgreSQL reconstrói o trabalho que o processo estava realizando.

3.4 Contexts isolados

Phoenix Contexts são a principal unidade de organização do domínio. Context não é sinônimo de um arquivo gigante.

Context root
→ operações sobre o conceito principal

Capability modules
→ capacidades públicas específicas

Internal modules
→ implementação privada

Outros contexts não acessam schemas, queries ou internals diretamente.

4 As três unidades fundamentais: Package, Integration e Run

Muitos sistemas misturam código reutilizável, configuração de cliente e execução. O Leafcutter separa essas responsabilidades.

4.1 Integration Package

Package é código versionado.

```
Integration Package
├─ manifest.json
├─ JSON Schemas
├─ Transformations
├─ optional Enrichments
├─ Interceptors
└─ dependency versions
```

Ele descreve o que a integração faz, mas não contém credenciais ou URLs de produção.

4.2 Integration

Integration é uma instância configurada do Package para um cliente e ambiente.

```
Acme / Homologation
Customer Distribution
Package: customer_distribution@1.3.0
Source Connection: ERP HML
Destination Connection: CRM HML
```

Outra Integration pode usar o mesmo Package em produção com Connections diferentes.

4.3 Run

Run é uma execução concreta.

Quando começa, o Leafcutter resolve:

- Package Version;
- Contract Versions;
- dependency versions;
- Connection references;
- defaults e overrides;
- batching;
- concurrency;
- Trigger metadata.

O resultado é um Run Snapshot imutável.

```
Package defaults
+ Integration overrides
+ resolved versions
= Run Snapshot
```

Se alguém alterar a Integration enquanto o Run está ativo, o Run não muda.

5 Catalog

O Catalog permite descobrir e versionar building blocks reutilizáveis.

```
Catalog
├── Connectors
│   ├── Official Connectors
│   └── Custom Connectors
├── Operations
├── Contracts
└── Integration Packages
```

O Catalog não executa o fluxo. Ele controla identidade, versões, dependências e disponibilidade.

Um Official Connector é mantido pela plataforma. Um Custom Connector representa um sistema específico. Para APIs REST simples, o Generic HTTP Connector evita criar código novo.

6 Contracts com JSON Schema

Tudo que entra ou sai de uma integração é tratado como JSON ou como uma representação compatível com JSON dentro do Elixir.

JSON Schema Draft 2020-12 define a estrutura válida.

```
untrusted external payload
↓
source JSON Schema / JSV
↓
trusted Elixir map
↓
Transformation
↓
candidate output
↓
destination JSON Schema / JSV
↓
valid payload
```

6.1 Por que validar as duas pontas

O Source Contract protege o runtime contra dados externos inesperados.

O Destination Contract protege o sistema externo contra bugs no próprio Package.

Se a Transformation esquecer um campo obrigatório, a falha ocorre antes do HTTP e é registrada como erro de validação.

6.2 Contratos versionados

Contracts são imutáveis. Um Run histórico continua explicável mesmo que o contrato atual tenha evoluído.

6.3 Compilação

O schema é compilado uma vez e reutilizado. Não interpretar o mesmo JSON Schema para cada Record.

```
Contract Version
→ compiled JSV validator
→ cache
→ many validations
```

Referências remotas são resolvidas e congeladas antes da execução. Um Run nunca depende da internet para compreender seu contrato.

7 Connector, Operation e Transport

Esta separação impede que o runtime genérico seja contaminado por detalhes de HTTP ou de um fornecedor específico.

```
Connector
→ understands the external system

Operation
→ understands one action

Transport
→ understands the protocol
```

7.1 Connector

Um Salesforce Connector pode conhecer:

- OAuth;
- base URL conventions;
- default headers;
- common errors;
- rate-limit headers;
- Operations disponíveis.

Ele não conhece Acme nem a Transformation de um Package.

7.2 Read Operation

Uma Operation de leitura normaliza paginação:

```
fetch(config, cursor)
```

retorna:

```
{:ok,  
  %{  
    records: [...],  
    next_cursor: next_cursor,  
    done?: false,  
    metadata: %{}  
  }}
```

O Source Broadway não precisa saber se a API usa `page`, `offset`, `cursor` ou `next_url`.

7.3 Write Operation

A Operation de escrita recebe um batch já transformado e validado. Ela preserva resultado por item.

```
{:ok,  
  [  
    %{status: :success, destination_identity: "CRM-9001"},  
    %{status: :error, error: operation_error}  
  ]}
```

Isso é necessário porque APIs de batch podem aceitar 99 registros e rejeitar apenas um.

7.4 Transport

HTTP é o primeiro Transport. A arquitetura não pressupõe que será o único.

No futuro, Database, SFTP ou outros Transports podem implementar o mesmo modelo sem alterar Run, Record ou Delivery.

8 Source Identity, Payload Hash e IdentityMapping

Três conceitos parecidos precisam permanecer separados.

8.1 Record ID

Identifica a ocorrência interna dentro de um Run.

8.2 Source Identity

Identifica a entidade estável no sistema de origem.

Uma Operation conhecida sabe extrair sua identity. Generic HTTP pode declarar algo simples:

```
"identity": "id"
```

ou chave composta:

```
"identity": ["company_id", "customer_id"]
```

8.3 Payload Hash

Representa o conteúdo atual.

```
same Source Identity  
+ different Payload Hash  
= same entity, changed data
```

8.4 IdentityMapping

Relaciona a mesma entidade entre sistemas.

```
ERP Customer 42
├─ CRM Customer CRM-9001
└─ Billing Customer BILL-710
```

Isso permite update/upsert no próximo Run e reduz duplicatas.

9 Transformation

Transformation monta o payload de negócio.

Ela pode:

- renomear campos;
- combinar strings;
- fazer cálculos;
- aplicar regras condicionais;
- criar estruturas aninhadas;
- normalizar datas;
- decidir `:skip`;
- produzir vários payloads.

Retornos:

```
{:ok, payload}      1 -> 1
{:ok, [payloads]}   1 -> N
:skip               1 -> 0
{:error, reason}    controlled failure
```

Não suporta N->1, joins ou aggregation stateful. Esses problemas exigem outra primitive futura.

10 Enrichment

Às vezes o payload precisa de uma consulta externa antes da Transformation.

Exemplos:

- risk score;
- currency rate;
- inventory lookup;
- external code resolution;
- address metadata;
- product metadata.

Essa request on-the-fly não deve ficar escondida dentro de `transform/2`.

```
validated source
  ↓
Enrichment preparation
  ↓
Connector Operation side effect
  ↓
persisted enrichment result
  ↓
Transformation
```

Enrichment é durável e retryable. No runtime, ele é uma Broadway pipeline opcional dentro da árvore do Run.

Se B e C precisam do mesmo resultado, o Enrichment acontece uma vez antes do fan-out. Enrichment específico por destination pode ser adicionado depois sem alterar a primitive.

11 Interceptor

Interceptor adapta a comunicação, não o dado de negócio.

Casos:

- correlation ID;
- custom header;
- query parameter;
- signing;
- URL adjustment;

- tracing metadata.

```
valid destination payload
↓
Operation builds request
↓
explicit Interceptors
↓
Transport
```

Na primeira arquitetura, Interceptor não modifica o body depois da validação do Destination Contract.

Interceptors globais invisíveis são evitados. O Package declara explicitamente quais Interceptors participam de cada destination.

12 O modelo de execução

12.1 Run supervision tree

```
RunDynamicSupervisor
├─ RunSupervisor <run_id>
│   ├── RunCoordinator
│   ├── SourceBroadway
│   ├── optional EnrichmentBroadways
│   └─ DestinationBroadways
│       ├── CRM
│       └─ Billing
```

A árvore de supervisão representa os principais lifecycles e failure domains do Run.

12.2 RunCoordinator

O Coordinator controla:

- start;
- pause;
- resume;
- cancel;
- conclusão da source;
- conclusão/falha de destinations;

- conclusão do Run.

Ele não recebe uma mensagem por Record. Não faz HTTP. Não transforma payload. Não é gargalo do data plane.

12.3 Broadway como data plane

Broadway já oferece producer, processors, batchers, demand, backpressure e supervision. O Leafcutter não cria managers e workers próprios para duplicar essas responsabilidades.

Source, Enrichment e cada destination possuem pipelines independentes quando necessário.

13 Fan-out durável

O Leafcutter persiste o trabalho antes de entregá-lo aos destinations.

Para mil Records e dois destinos:

```
1,000 Records
2,000 Deliveries
1 Checkpoint advancement
```

são persistidos em batch na mesma transação.

```
Source Broadway
  ↓ validate page
build Records + Deliveries
  ↓
Postgres transaction
├─ insert Records
├─ insert Deliveries
└─ advance Checkpoint
  ↓ COMMIT
```

Se a transação falha, o checkpoint não avança.

13.1 Por que não usar fila externa primeiro

Broadway já controla demand e batching. Bulk inserts reduzem pressão. Uma fila antes do Postgres absorveria bursts, mas adicionaria infraestrutura e não eliminaria a necessidade de persistir o mesmo volume.

A primeira versão mede Postgres. Kafka, RabbitMQ ou outro broker entram somente se as métricas demonstrarem que o durable backlog em Postgres é o gargalo dominante.

Delivery permanece parte do domínio mesmo se o mecanismo de transporte mudar no futuro.

14 Destination pipelines

Cada destination consome suas Deliveries de forma independente.

```
Postgres
├── CRM Deliveries
└── Billing Deliveries

CRM Broadway      Billing Broadway
  ↓                ↓
transform          transform
validate           validate
batch 100          batch 500
send CRM           send Billing
```

Um destination lento acumula backlog próprio. Os demais continuam.

14.1 Claim

O Producer faz claim atômico de Deliveries disponíveis em transação curta, por exemplo usando uma estratégia baseada em `FOR UPDATE SKIP LOCKED`.

O lock não permanece aberto durante a request externa.

14.2 Retry

Uma Delivery retryable retorna a:

```
status = pending
available_at = future timestamp
attempt_count += 1
```

O Producer só busca trabalho disponível.

Oban não precisa representar milhões de Deliveries. Ele permanece responsável por schedules, notifications, maintenance e outros trabalhos futuros duráveis.

15 Error model

Operations traduzem erros externos para uma taxonomia pequena:

```
validation
authentication
rate_limited
timeout
temporary
permanent
```

Decisões:

```
validation      no automatic retry
authentication  blocked until config/secret changes
rate_limited    retry later
timeout         retry
temporary       retry
permanent       no automatic retry
```

Partial batch success é preservado por Delivery. Uma request 429 para o batch inteiro não confirma nenhum item.

16 At-least-once

O Leafcutter assume que trabalhos podem ser repetidos.

Exemplo crítico:

```
CRM accepted request
↓
node crashed before local commit
↓
Leafcutter cannot prove completion
↓
retry
```

O sistema não promete exactly-once universal porque não controla a transação do CRM.

O efeito pode se tornar efetivamente único por:

- idempotency key;
- upsert;

- Source Identity;
- IdentityMapping;
- destination-specific support.

17 Checkpoint

Checkpoint é estado durável, não processo OTP.

O Source Producer mantém cursor atual em memória. PostgreSQL guarda o último cursor seguro.

```
runtime cursor = page 47
safe checkpoint = page 45
```

Crash pode repetir 46 e 47. Isso é consistente com `at-least-once`.

Não existe `CheckpointSupervisor` sem um lifecycle concreto que o justifique.

18 Run ownership e recovery entre nodes

18.1 Um Run, um node

Na primeira arquitetura, a árvore inteira de um Run vive em um node.

```
Node A
├─ Run 9182
│   ├── Coordinator
│   ├── Source
│   ├── Enrichment
│   ├── CRM
│   └─ Billing
```

Não espalhar branches individuais pelo cluster cedo.

18.2 Heartbeat por node

```
runtime_nodes
node_a heartbeat_at
node_b heartbeat_at
```

Run guarda:

```
owner_node
generation
```

18.3 Recovery

Se Node A desaparece e seu heartbeat expira, Node C pode claimar atomicamente o Run, incrementar generation e carregar snapshot/checkpoint.

```
owner_node: node_a → node_c
generation: 4 → 5
```

Generation é fencing token. Escritas críticas de um owner antigo com generation 4 são rejeitadas.

Distributed Erlang oferece `nodedown` como sinal rápido, mas PostgreSQL continua sendo a autoridade.

Registry é local ao node e serve para localizar processos locais por Run ID.

19 O cluster BEAM

19.1 Infraestrutura inicial

```
Internet
  ↓
Load Balancer
  ↓
BEAM Node A  BEAM Node B  BEAM Node C
      \          |          /
          PostgreSQL
```

Todos os nodes são homogêneos inicialmente. A mesma release contém Core, Connectors, Runtime e API.

19.2 Distributed Erlang

Usado para:

- cluster PubSub;
- node membership;
- comunicação operacional;
- nodeup/nodedown.

Não usado como:

- banco;
- fila durável;
- ownership definitivo;
- motivo para criar distributed locks customizados.

Não usar `:global` , `:pg` , Horde ou CRDTs para ownership inicial de Runs.

19.3 Evolução

Somente após métricas:

```
homogeneous nodes
→ API/runtime specialization
→ object storage
→ package isolation
→ analytics store
→ external queue if justified
```

20 Connections e Secrets

20.1 Connection

Connection descreve acesso não sensível a um sistema em um Environment:

- Connector;
- base URL;

- account/region;
- timeouts defaults;
- referência ao Secret.

20.2 Secret

Guarda:

- API keys;
- client secrets;
- refresh tokens;
- passwords;
- certificates.

Secrets são versionados, rotacionáveis e protegidos por RBAC. OAuth refresh que produz novo token é persistido de forma durável.

Nenhum secret entra em manifest, logs, AuditEvent ou Run Snapshot público.

21 Environments, homologação e promoção

Package Versions são imutáveis. Environment controla configuração operacional.

```
Package 1.4.0
  ↓ HML deployment
homologation Runs
  ↓ approval
promotion
  ↓ PROD deployment
```

Promoção não copia credenciais. HML e PROD usam suas próprias Connections e Secrets.

Rollback seleciona uma versão anterior e registra AuditEvent.

Homologation relaciona:

- versão testada;
- evidências/Runs;
- approver;
- target environment;

- resultado;
- timestamps.

22 RBAC

Permissão é a primitive; Role é um agrupamento.

```
who
→ User or ServiceAccount

what
→ Permission

where
→ Organization + Environment
```

Exemplos:

```
integration.read
integration.write
integration.run
integration.approve
integration.promote
run.retry
run.cancel
secret.rotate
payload.read
audit.read
```

É possível ver status sem poder ler raw payloads.

Autorização não fica apenas no controller. Operações privilegiadas recebem actor/scope e aplicam policy na API do domínio.

23 API-first

O Leafcutter será operável sem frontend.

OpenAPI

- platform documentation
- Postman Collection
- future SDKs

Postman não possui conhecimento exclusivo. SDKs só entram quando a API estiver estabilizada.

A API deve representar workflows completos:

```
create Organization
→ create Environments
→ create Connections
→ publish Package Version
→ create Integration in HML
→ execute
→ inspect
→ approve
→ promote
→ execute in PROD
```

Inbound Endpoints entram como sources HTTP em uma release posterior. API Management completo é outro estágio do produto.

24 Monitoring, observabilidade e auditoria

24.1 Execution Monitoring

Consulta dados de domínio:

Run

- Records
- Deliveries
- Attempts
- Enrichments
- ExecutionEvents

24.2 Platform Observability

Mede a plataforma:

- throughput;
- Broadway demand;
- backlog;
- DB latency;
- node health;
- memory;
- HTTP latency;
- rate-limit pressure.

24.3 Attempt

Representa uma tentativa concreta de comunicação externa.

24.4 ExecutionEvent

Representa fatos relevantes do Run, não cada mensagem interna.

24.5 AuditEvent

Representa ações humanas/administrativas: promotion, rollback, secret rotation, manual retry, permission change.

24.6 Notifications

Notification Rules reagem a eventos e usam Oban para entrega durável a Channels/Recipients. PubSub sozinho não garante notificação.

25 Storage e retenção

O modelo lógico não obriga armazenamento quente eterno.

PostgreSQL
→ operational truth and queryable metadata

Object Storage future
→ large immutable payloads

Analytics Store future
→ large historical aggregation

Na primeira versão, payloads podem ficar em JSONB por simplicidade. O modelo deve separar metadata de content para permitir externalização futura.

Payload access possui permissão específica. Request/response bodies antigos podem ter SLA assíncrono para audit export.

26 Phoenix Contexts

A proposta inicial de Context Map é:

Organizations
Catalog
Connections
Integrations
Executions
Notifications
Audit

Essa proposta ainda será ratificada um context por vez.

26.1 Por que não um context por tabela

Run, Record, Delivery, Attempt e Checkpoint pertencem ao mesmo lifecycle de execução. Separá-los em cinco contexts criaria cerimônia e transações difíceis.

26.2 Por que Executions não vira god module

A facade é segmentada:

```
Executions.Runs.start(...)
Executions.Runs.pause(...)
Executions.Deliveries.retry(...)
Executions.Attempts.list(...)
Executions.Recovery.claim(...)
```

Schemas e implementação continuam internos.

27 Apps da umbrella

Proposta inicial:

```
apps/
├─ leafcutter_core
├─ leafcutter_connectors
├─ leafcutter_runtime
└─ leafcutter_api
```

27.1 Core

Repo, contexts de negócio, PubSub e contracts públicos mínimos.

27.2 Connectors

Connector/Operation/Transport behaviours e implementações.

27.3 Runtime

Executions, Registry, DynamicSupervisor, Broadway, heartbeat e recovery.

27.4 API

Phoenix Endpoint, controllers, auth, OpenAPI e Inbound APIs futuras.

Tudo sobe numa release inicialmente.

A divisão exata, incluindo Oban, PubSub, Repo e build de `packages/`, ainda será ratificada antes de criar as apps.

28 Documentação como parte da arquitetura

Todo código e documentação in-code são em inglês.

Módulos públicos relevantes incluem:

```
@moduledoc
@doc
@typedoc
@spec
named error contracts
```

A arquitetura fora do código permanece em pt-BR.

ExDoc é uma documentação técnica viva. Comentários explicam por que uma decisão incomum existe.

29 Harness e continuidade

A conversa não é a memória canônica. O repositório é.

```
AGENTS.md
→ regras permanentes para agentes

ADRs
→ decisões duráveis

Architecture docs
→ modelo e intenção

CURRENT.md
→ estado atual e próxima tarefa

Code + tests
→ comportamento real
```

O desenvolvedor é o autor principal. Codex atua por padrão em Guide Mode, Review Mode ou Debug Mode.

Uma nova sessão começa lendo:

1. AGENTS.md ;
2. CURRENT.md ;

- 3. ADRs relevantes;
- 4. código e testes atuais.

30 Roadmap

30.1 V0 - Foundation

Documentação, contexts, apps, Repo, CI, ExDoc e package spec.

30.2 V1 - Reliable HTTP core

Generic HTTP, Packages, Contracts, one Source -> N Destinations, durable fan-out, Broadway, manual Runs, Attempts, IdentityMapping e API.

30.3 V1.x - Reliability

Retries avançados, schedules, Enrichments, notifications, multi-node recovery e audit.

30.4 V2 - Governance

RBAC, homologação, promotion/rollback, payload access e secret rotation.

30.5 V3 - Ecosystem

Official/Custom Connectors, package tooling, dependency resolution, registry e OpenAPI import.

30.6 V4 - Inbound e multi-transport

Inbound APIs, Database/SFTP transports, Codecs e object storage.

30.7 V5+

Package isolation, specialized nodes, analytics, external queue somente se justificado, multi-source e SDKs após estabilização.

31 O que ainda precisa ser ratificado

A baseline está sólida, mas os próximos passos são deliberadamente um por vez:

1. Context Map definitivo.
2. Ownership de conceitos/tabelas.
3. APIs públicas/capability modules.
4. Apps da umbrella e dependency graph.
5. Repo, PubSub e Oban placement.
6. Package compilation strategy.
7. Package Manifest JSON Schema v1.
8. Ecto schemas, states, constraints e indexes.
9. OpenAPI surface.
10. Runtime details de pause/cancel/shutdown/backlog limits.

32 Conclusão

O Leafcutter usa a BEAM onde ela é forte sem transformar distribuição em objetivo próprio.

```
OTP
→ lifecycle and fault isolation

Broadway
→ data flow, demand and batching

PostgreSQL
→ durable truth and recovery

Oban
→ durable future work

PubSub
→ ephemeral propagation

Phoenix
→ API and operational boundary

Ecto
→ constraints and persistence

JSON Schema + JSV
→ trustworthy data boundaries
```

A arquitetura procura manter duas qualidades ao mesmo tempo:

- um runtime capaz de lidar com concorrência, falhas e volume;
- um código que um desenvolvedor consiga abrir, ler e compreender sem depender de um agente para explicar cada camada.

Essa combinação é o critério central do Leafcutter.